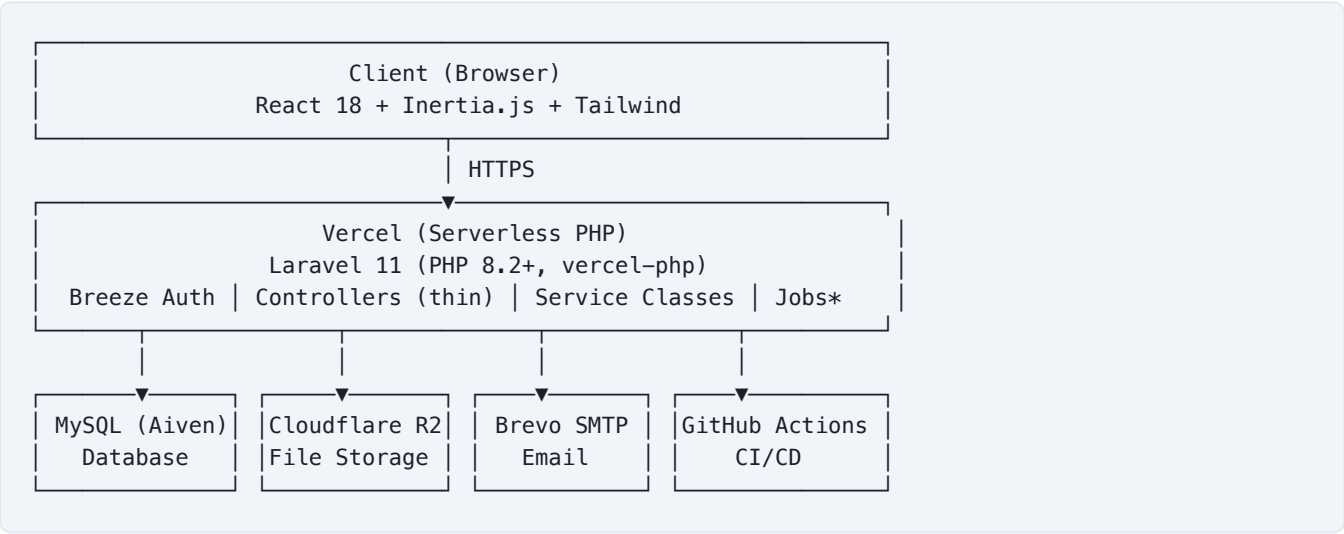


Belia Hub

System Architecture Report

Platform digital organisasi belia — pengurusan keahlian & marketplace servis · Zero-Cost Serverless Deployment · Dokumen rujukan: ARCHITECTURE.md, FUNCTIONAL_SPEC.md, DATABASE.md · Dijana: 27 Julai 2026

1. High-Level Architecture



* Jobs run synchronously or via Vercel cron (see Serverless Constraints).

2. Tech Stack

Layer	Technology	Notes
Backend	Laravel 11.x, PHP 8.2+	<code>declare(strict_types=1);</code> everywhere
Frontend	React 18 + Inertia.js	Functional components + hooks only
Styling	Tailwind CSS	—
Auth	Laravel Breeze (React stack)	Session-based
Database	MySQL on Aiven (DBaaS)	SSL connection required
Runtime	Vercel serverless (<code>vercel-php</code>)	Ephemeral, read-only filesystem
File storage	Cloudflare R2	Via Laravel <code>Storage::disk('s3')</code>
Email	Brevo SMTP (prod), Mailtrap (dev)	—
CI/CD	GitHub Actions → Vercel	Lint, test, deploy

3. Serverless Constraints (CRITICAL)

Vercel's filesystem is ephemeral and read-only — non-negotiable rules

1. **NEVER** save user uploads with `storage_path()` or `public_path()`.
2. **ALWAYS** use `Storage::disk('s3')` (Cloudflare R2) for all uploads: reference materials, delivery files, invoices, QR images.
3. **NO** long-running `queue:work` daemons. Priority order: synchronous processing (`QUEUE_CONNECTION=sync`) for emails/PDFs; Vercel cron jobs hitting a signed artisan endpoint for scheduled tasks.
4. Cache/session drivers: use `database` or `cookie`, never `file`.
5. Compiled views: pre-compile at build time or point `view.compiled` at `/tmp`.
6. Logs: `LOG_CHANNEL=stderr` (Vercel captures stdout/stderr).

4. Application Layers

```
Route → Middleware (auth, role) → Controller (thin)
      → FormRequest (validation)
      → Service class (business logic)
      → Eloquent Model (data)
      → Inertia::render(...) / redirect
```

- **Controllers:** thin, no business logic. One resource controller per module.
- **FormRequests:** ALL validation lives here. Authorization checks per role.
- **Service classes:** `app/Services/` — e.g. `SequenceService`, `OrderService`, `InvoiceService`, `AttendanceService`.
- **Eloquent only:** no raw SQL. Relationships defined on models.

5. RBAC Design

Four roles stored on `users.role` (enum): `superadmin`, `provider`, `member`, `client` (default for new registrations).

- Middleware: `role:superadmin`, `role:superadmin,provider` etc.
- Policies for record-level checks (e.g. user can only view own orders; provider can only update assigned orders).

Route group	Access
/ , /servis , /terma , /privasi	Public (landing, catalog, terms, privacy)
/admin/*	Superadmin
/provider/*	Provider (+ superadmin)
/dashboard , /orders	All authenticated verified users (members AND clients)
/events , attendance endpoints	Members/providers only (clients blocked)

Membership upgrade path: client applies → superadmin approves → role member + BH- ID generated.

6. Planned Directory Structure (key paths)

```
app/  
  Enums/           UserRole, OrderStatus, PaymentType, FileCategory,  
                   EventStatus, SequenceType  
  
  Http/  
    Controllers/   Admin/, Provider/, Member/  
    Middleware/     EnsureUserHasRole.php, EnsureUserIsActive.php  
    Requests/       one FormRequest per action  
  Models/          User, Skill, Service, ServiceOrder, OrderFile, OrderComment,  
                   Payment, Payout, Invoice, Event, Attendance, Sequence  
  
  Policies/  
  Services/        SequenceService (membership/order/invoice IDs),  
                   OrderService, InvoiceService (invoice + receipt PDFs),  
                   AttendanceService, R2UploadService  
  
resources/js/  
  Layouts/         AuthenticatedLayout, AdminLayout, GuestLayout  
  Pages/           Admin/, Provider/, Member/, Auth/  
  Components/  
  routes/          web.php, auth.php  
  database/        migrations/, seeders/, factories/  
  docs/            documentation  
  vercel.json      Vercel runtime config
```

7. Environment Variables

```
APP_KEY, APP_URL  
APP_TIMEZONE=Asia/Kuala_Lumpur  (critical: attendance windows & payment dates;  
                                Vercel defaults to UTC)  
DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD  (Aiven, MYSQL_ATTR_SSL_CA)  
AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY                (R2 keys)  
AWS_DEFAULT_REGION=auto  
AWS_BUCKET, AWS_ENDPOINT, AWS_USE_PATH_STYLE_ENDPOINT=true  
MAIL_MAILER=smtp, MAIL_HOST (Brevo), MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD  
QUEUE_CONNECTION=sync  
SESSION_DRIVER=database  
CACHE_STORE=database  
LOG_CHANNEL=stderr  
ORG_BANK_NAME, ORG_BANK_ACCOUNT_NO, ORG_BANK_ACCOUNT_NAME  (payment instructions  
                                                             shown to clients)
```

App-level settings live in `config/beliahub.php`: locality list (dropdown options), stale-order threshold (14 days), attendance grace period, file upload limits.

8. CI/CD Pipeline (GitHub Actions)

1. **On PR:** `composer install` → Pint (lint) → PHPUnit/Pest tests → `npm run build` check.
2. **On merge to main:** Vercel auto-deploy (Git integration) or `vercel deploy --prod` step.
3. Migrations run via deploy hook or manual `php artisan migrate --force` against Aiven.

9. Key Flows

Registration → Membership (open registration, admin-approved membership)

User registers (Breeze, default role `client`) → email verified → can order services. Optionally applies for membership (at registration or later) → superadmin approves → role upgraded to `member` → `SequenceService` generates `BH-{YEAR}-{NNNN}` (DB transaction + `lockForUpdate()` on sequences row) → welcome email (BM) sent via Brevo (sync). Clients never get a `BH-` ID and cannot access events.

Service Order (money in: client → organization)

Member/client picks service from catalog (types & prices managed by superadmin; order stores unit price snapshot + quantity) → order created pending with `total = unit_price × quantity`, `deposit = total × 0.5` → client uploads reference files + payment proof → R2 → superadmin verifies proof, records deposit payment (receipt PDF auto-emailed), confirms + assigns provider → invoice PDF → R2 → emailed to client → client/provider communicate via order comment thread → provider uploads delivery files, marks `completed` → superadmin records balance payment.

Payout (money out: organization → provider)

Order completed & fully paid → superadmin records payout to assigned provider (manual amount, method, date) → provider views own payout history. Dashboard reports gross collection – refunds – payouts = net revenue.

Event Attendance (rotating QR)

Superadmin creates event → during event, admin opens "Display QR" page showing a short-lived signed URL QR (~5 min validity, auto-refreshes) → member scans → hits signed + throttled endpoint → attendance row created (unique per user+event, within event time window).